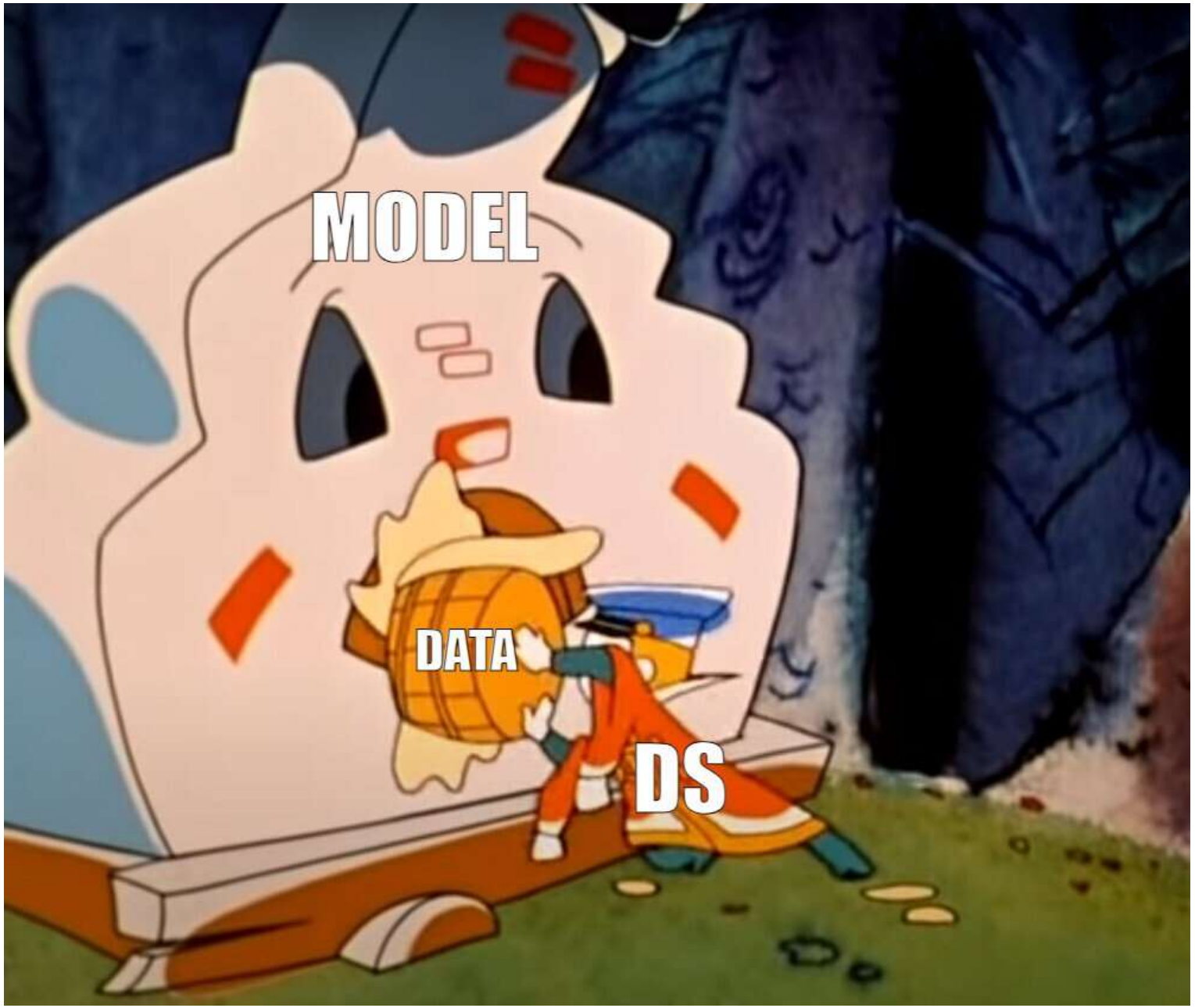
Тверской
государственный
технический
университет

Интеллектуальные информационные системы

Отбор признаков и понижение размерности

2026 г.



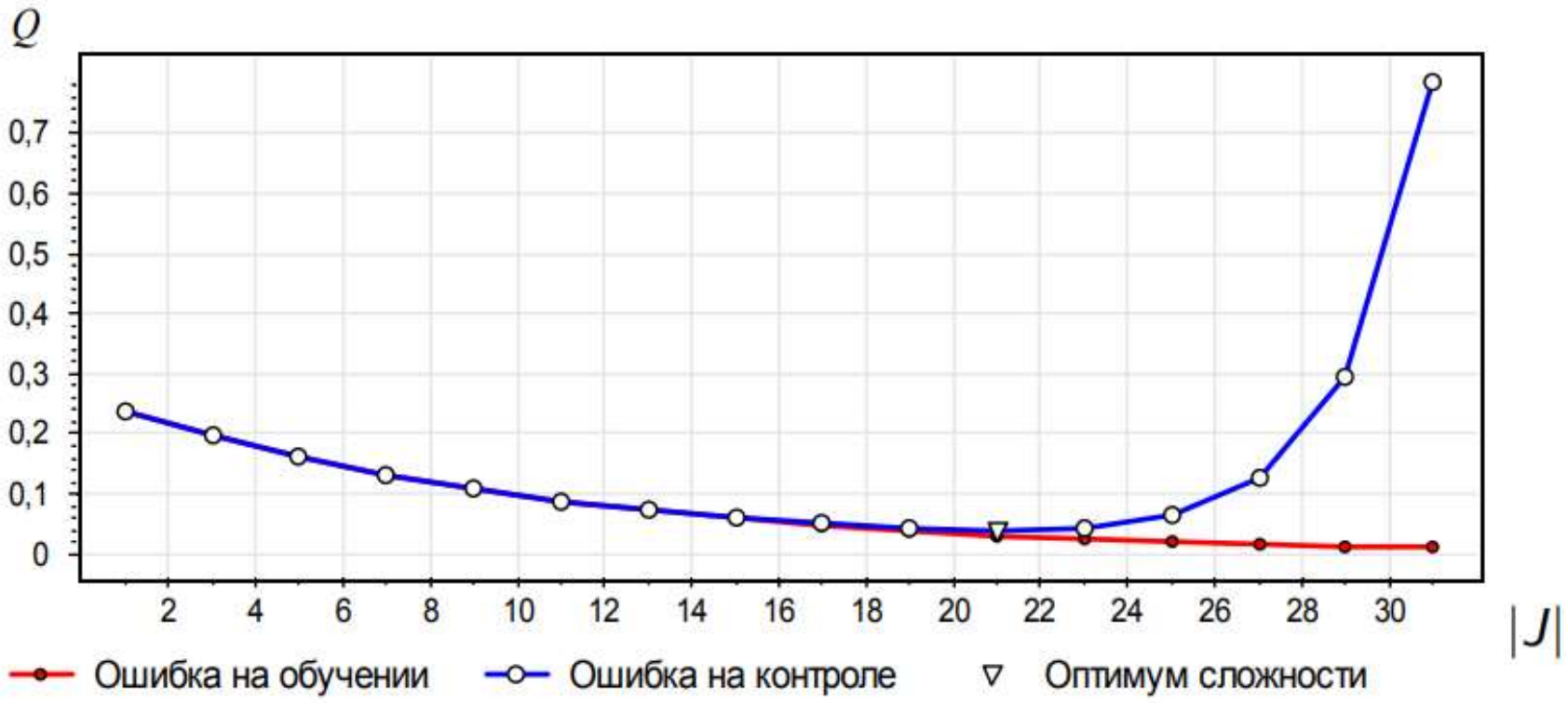
«Засунем всё в модель – и так сойдет»

Мотивация отбора признаков

- Признак может быть *неинформативным* – признак никак не влияет на целевую переменную
- Признак может быть *зависимым* – информация о признаке содержится в других признаках
- Признак может быть сильно *зашумленным* – его использование может повышать риск ошибки
- Минимизация числа признаков позволяет уменьшить затраты *времени и ресурсов* – особенно важно при наличии ограничений на инференс модели
- *Понижение размерности* может уменьшать переобучение
- Меньшее число признаков повышают интерпретируемость модели

Критерий отбора признаков

Внутренний критерий и внешний критерий:



Источник: http://www.machinelearning.ru/wiki/images/d/de/Voron-ML-Quality-slides.pdf

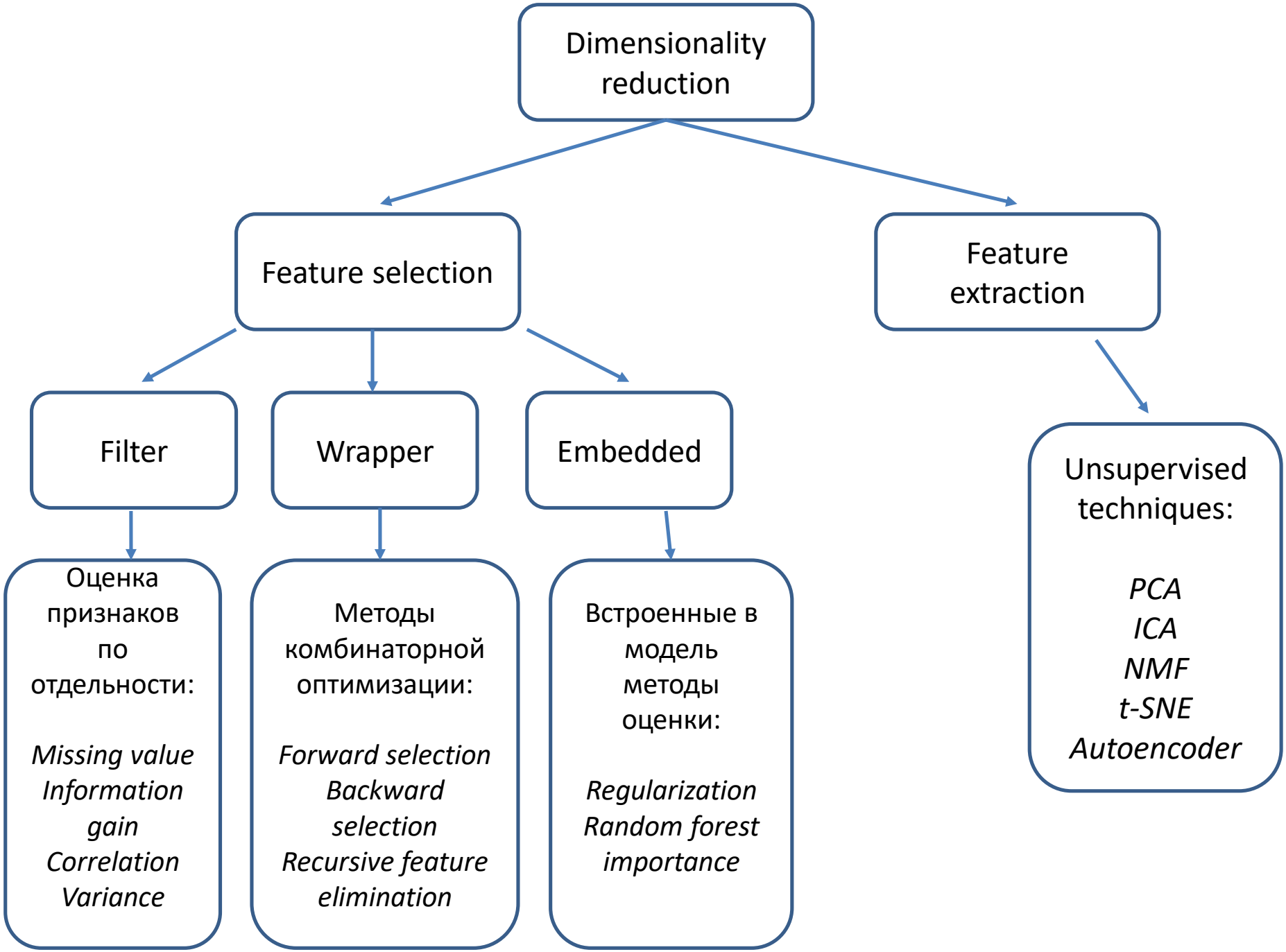
Отбор признаков без утечки данных

Отбор признаков правильно выполнять на этапе обучения модели. Нельзя отбирать на всех данных до разделения данных - отбор по всему датасету использует сведения о контрольной выборке -> утечка данных, может завысить оценку качества.

TRAIN: обучаем заполнение пропусков, масштабирование, отбор, PCA и т.д. Статистики рассчитываем только по обучающей выборке.

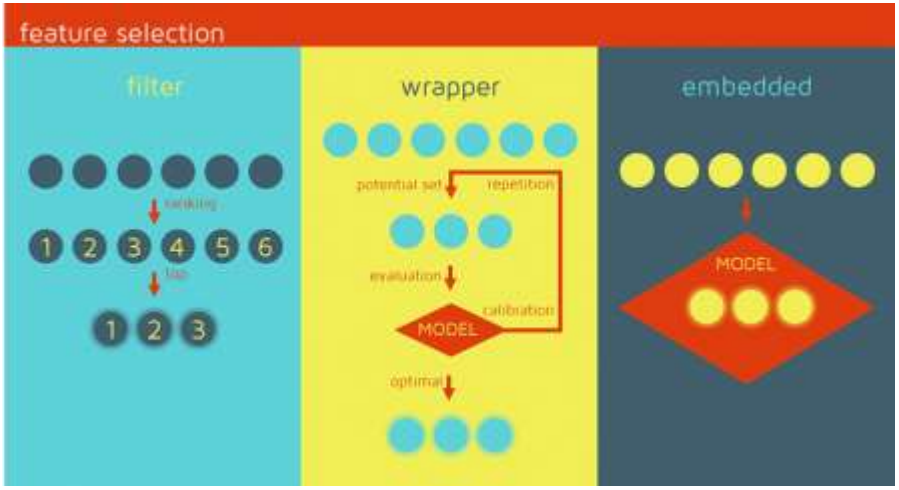
VALID: применяем готовые преобразования через transform и вычисляем метрику. Не вызываем fit или fit_transform на контрольной выборке.

Test не участвует в выборе признаков - используем для итоговой оценки.


Dimensionality reduction
Feature selection
Feature extraction
Filter
Wrapper
Embedded
Unsupervised techniques:
PCA
ICA
NMF
t-SNE
Autoencoder
Оценка признаков по отдельности:
Missing value
Information gain
Correlation
Variance
Методы комбинаторной оптимизации:
Forward selection
Backward selection
Recursive feature elimination
Встроенные в модель методы оценки:
Regularization
Random forest importance

- *Feature selection*

поиск оптимального подмножества признаков из исходного множества признаков



- *Feature extraction*

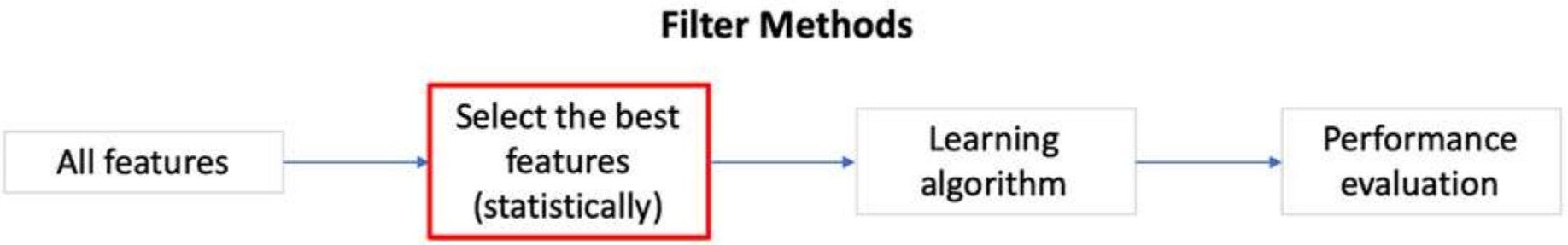
преобразование исходного признакового пространства в подпространство меньшего размера с максимальным сохранением информации



Filter methods

- *Методы фильтрации признаков* – основаны на оценке информативности каждого признака, с последующей сортировкой и выбором k лучших. Оценивают свойства признаков без обучения модели – быстрый предварительный этап отбора.

- Не учитывают зависимости между признаками
- Не всегда позволяют оценить «полезность» признака, больше подходят для того, чтобы найти явно «бесполезные» признаки
- Малая вычислительная сложность
- Не зависят от модели
- Чувствительны к выбору критерия отбора (пороговые значения для околонулевой дисперсии, коэффициентов корреляции и т.д.)



- *Missing value ratio* – оцениваем отношение количества пропущенных значений к общему числу наблюдений для каждого признака

✓ Интуитивно понятный метод

✓ Признаки с большим количеством пропущенных значений лучше выкидывать, чем заполнять (или нет)

× Чувствительность к значению порога

× Пропущенные значения могут быть связаны с целевой переменной

× Выкидывать или заполнять? Заполнять можно статистиками или с помощью моделей, используя пропущенные значения в качестве целевой переменной

ID	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	count
AB101	1.0	0.0	0.0	1.0	9.84	14.395	81.0	NaN	16
AB102	1.0	NaN	0.0	NaN	9.02	13.635	80.0	NaN	40
AB103	1.0	0.0	NaN	1.0	9.02	13.635	80.0	NaN	32
AB104	NaN	0.0	NaN	1.0	9.84	14.395	75.0	NaN	13
AB105	1.0	NaN	0.0	NaN	9.84	14.395	NaN	16.9979	1
AB106	1.0	0.0	NaN	2.0	9.84	12.880	75.0	NaN	1
AB107	1.0	0.0	0.0	1.0	9.02	13.635	80.0	NaN	2
AB108	1.0	NaN	0.0	1.0	8.20	12.880	86.0	NaN	3
AB109	NaN	0.0	0.0	NaN	9.84	14.395	NaN	NaN	8
AB110	1.0	0.0	0.0	1.0	13.12	17.425	76.0	NaN	14

Variable	Missing value ratio
ID	0%
season	20%
holiday	30%
workingday	30%
weather	30%
temp	0%
atemp	0%
humidity	20%
windspeed	90%
count	0%

Источник: https://www.analyticsvidhya.com/blog/2021/04/beginners-guide-to-missing-value-ratio-and-its-implementation/

- *Variance threshold* – удаление признаков с дисперсией ниже заданного порога

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}\left(f_{i,j} - \overline{f_j}\right)^2$$

где $f_{i,j}$ – значение j – го признака на i – m объекте, $\overline{f_j}$ – среднее значение j – го признака

- ✓ Интуитивно понятный метод
- ✓ Легкая реализация
- × Чувствительность к значению порога
- × Чувствительность к шуму
- × Малая, но ненулевая дисперсия может указывать на связь с целевой переменной

- *Корреляция:*

- высокая степень корреляции между признаком и целевой переменной
- низкая степень корреляции между признаками

Можно использовать различные коэффициенты корреляции:

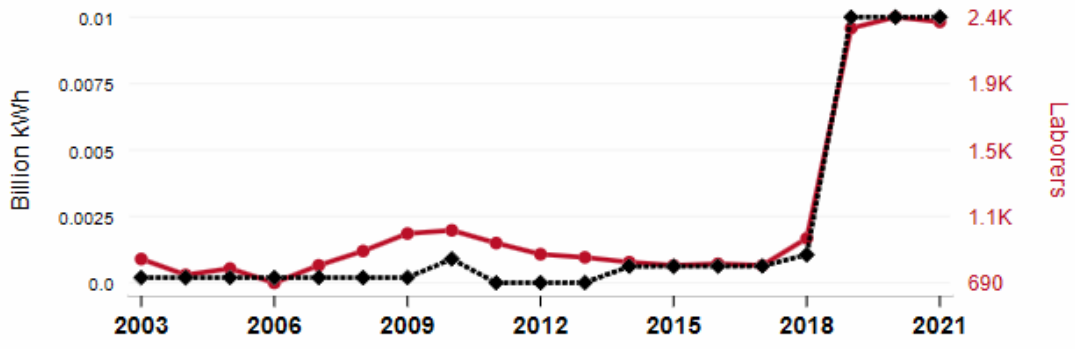
1. Коэффициент корреляции Пирсона (линейный коэффициент)
2. Коэффициент ранговой корреляции Спирмена
3. Коэффициент ранговой корреляции Кендалла
4. Коэффициент корреляции знаков Фехнера

Сам по себе факт корреляционной зависимости не дает основания утверждать, что одна из переменных предшествует или является причиной изменений, или то, что переменные вообще причинно связаны между собой, а не наблюдается действие третьего фактора

Solar power generated in Belize

correlates with

The number of fire inspectors in Florida



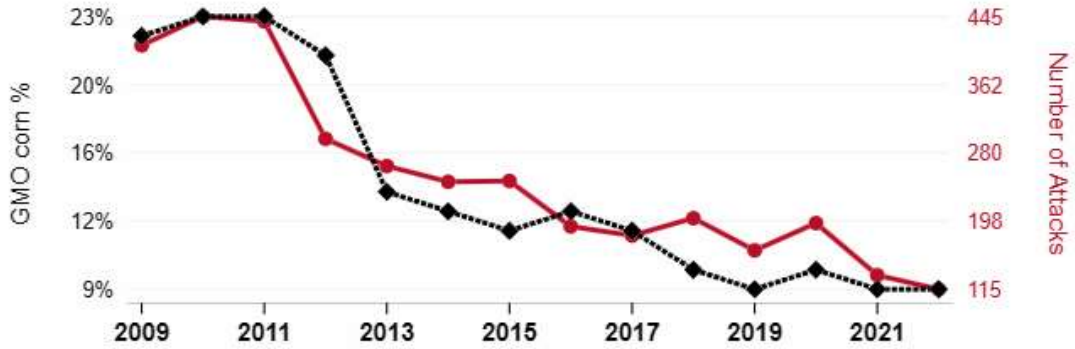
Total solar power generated in Belize in billion kWh · Source: Energy Information Administration

BLS estimate of fire inspectors and investigators in Florida · Source: Bureau of Larbor Statistics

GMO use in corn

correlates with

Pirate attacks globally



Percent of all corn planted in United States that is genetically modified to be herbicide-tolerant (HT), but not insect-resistant (Bt) · Source: USDA

Global Pirate Attack Count · Source: Statista

2009-2022, r=0.948, r^2=0.899, p<0.01 · tylervigen.com/spurious/correlation/2051

Источник: https://www.tylervigen.com/spurious-correlations

Одномерная оценка признаков

методы оценивают каждый признак отдельно

F-test (ANOVA) - сравнивает различия средних между классами с разбросом значений внутри классов. Чем заметнее различия относительно внутреннего разброса, тем выше F-статистика.

Отвечает на вопрос: различаются ли средние значения признака у объектов разных классов ?

Может не обнаружить связь, если средние одинаковы, но распределения отличаются по форме или разбросу.

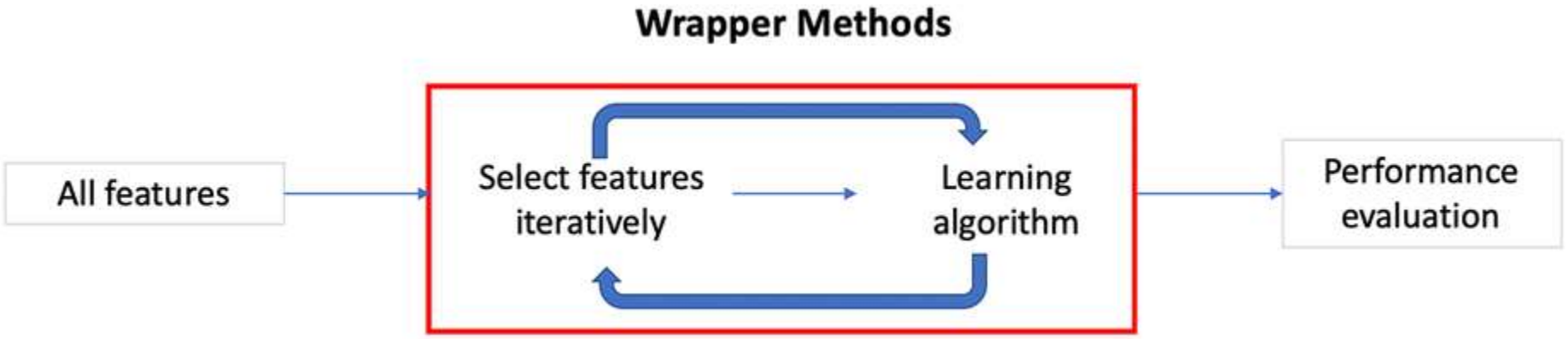
Chi square χ^2 – сравнивает фактическое число объектов в сочетаниях «категория признака × класс» с ожидаемым числом при отсутствии связи. Чем сильнее расхождение, тем выше χ^2. Непрерывный признак для такого анализа сначала разбивают на интервалы

Mutual information - измеряет статистическую зависимость признака и таргета, в том числе нелинейную. Теоретически равна нулю тогда и только тогда, когда они независимы. На практике вычисляется приближённо: результат зависит от объёма данных, типа признака и настроек метода оценки

Wrapper methods

- *Методы комбинаторной оптимизации* – переборные методы, основаны на выборе оптимального подмножества признаков из множества признаков

- Учитывают зависимости между признаками
- Часто обеспечивают хорошее качество
- Высокая вычислительная сложность (особенно при больших данных)
- Зависят от модели – подмножества перебираются для выбранной модели. Это подмножество не обобщается на все модели.



- *Алгоритм полного перебора (exhaustive feature selection)* – использует все возможные подмножества признаков для выявления лучшего подмножества

✓ Простота реализации

✓ Гарантированный результат

× Очень высокая вычислительная сложность – в общем случае $O(2^n)$

× Чем больше перебирается вариантов, тем больше переобучение

- *Прямые алгоритмы (step forward feature selection, алгоритмы жадного добавления)* – основаны на постепенном добавлении новых признаков в модель

1. Строится модель, содержащая один признак x_1
2. К признаку x_1 по очереди добавляются оставшиеся признаки x_j, строятся модели, содержащая два признака. На основе выбранной метрики оценивается качество всех моделей и выбирается лучшая модель из двух признаков
3. Итеративное повторение для N > 2 признаков
4. Критерии остановки:

- Минимальное улучшение качества
- Ограничение на количество признаков (максимальное)

- *Обратные алгоритмы (step backward feature selection, алгоритмы жадного удаления)* – основаны на постепенном удалении признаков из модели

1. Каждый признак x_i удаляется из множества признаков X, для оставшихся признаков строятся модели. На основе выбранной метрики оценивается качество всех моделей и выбирается лучшая модель, содержащая все признаки, кроме одного
2. Из подмножества X^{-1} по очереди удаляется по одному из оставшихся признаков x_j. Строятся модели модели на основе исходного множества без двух признаков. Оценивается качество всех моделей, выбирается лучшая модель, содержащая все признаки, кроме двух
3. Итеративное повторение для N > 2 признаков
4. Критерии остановки:

- Минимальное улучшение качества
- Ограничение на количество признаков (минимальное)

Recursive feature elimination

RFE (RFECV):

- обучить модель на текущем наборе признаков

- оценить важность признаков

- удалить наименее важные признаки

- обучаем модель и повторяем шаги

RFE использует ранжирование важностей текущей модели, не проверяет отдельно удаление каждого признака, как это делает backward selection

RFECV: число признаков выбирается по CV.

Permutation Feature Importance

Насколько ухудшится качество модели, если перемешать значения одного признака?

- случайно перемешиваем значения признака между объектами, сохраняя остальные признаки

- без переобучения модели повторно вычисляем метрику. Чем сильнее упало качество, тем важнее признак для этой модели

Перемешивание повторяем несколько раз, оценивая среднее падение и его разброс.

Коррелирующие признаки могут заменять друг друга, занижая индивидуальные оценки.

Алгоритм Boruta: сравнение с шумовыми признаками

Какие признаки устойчиво полезнее случайного шума?

1. Теневые признаки (Shadow Features):

Алгоритм создает копии всех существующих признаков и перемешивает их значения. Эти новые искусственные признаки называются "теневыми".

Поскольку их значения случайны, они не должны иметь никакой предсказательной силы. Теневые признаки добавляются к исходному набору данных, образуя расширенную матрицу признаков

2. Статистическое сравнение и итеративный отбор:

На полученном ранее наборе данных обучается модель, которая вычисляет важность каждого признака — как исходного, так и теневого.

Порогом важности для отбора в итоговую модель служит наивысший показатель важности среди всех теневых признаков. Максимальное значение Z статистики среди теневых признаков сокращенно обозначают как MZSA (maximum Z score among shadow attributes).

3. Каждый исходный признак сравнивается с этим порогом:
Если его важность статистически значимо выше порога (для каждого признака считается Z статистика), он получает "попадание" и считается важным. Если важность ниже, чем у лучшего теневого признака, он помечается как неважный. Этот процесс повторяется многократно. На основе количества попаданий в категорию за все итерации алгоритм окончательно классифицирует каждый признак на три категории:
- *Confirmed* - важные признаки
- *Tentative* - признаки, по которым алгоритм не смог принять однозначного решения.
- *Rejected* - неважные признаки

Преимущества: проверяет, является ли признак более информативным, чем случайный шум, а не просто более важным, чем другие признаки в конкретном наборе данных, в Permutation Importance.

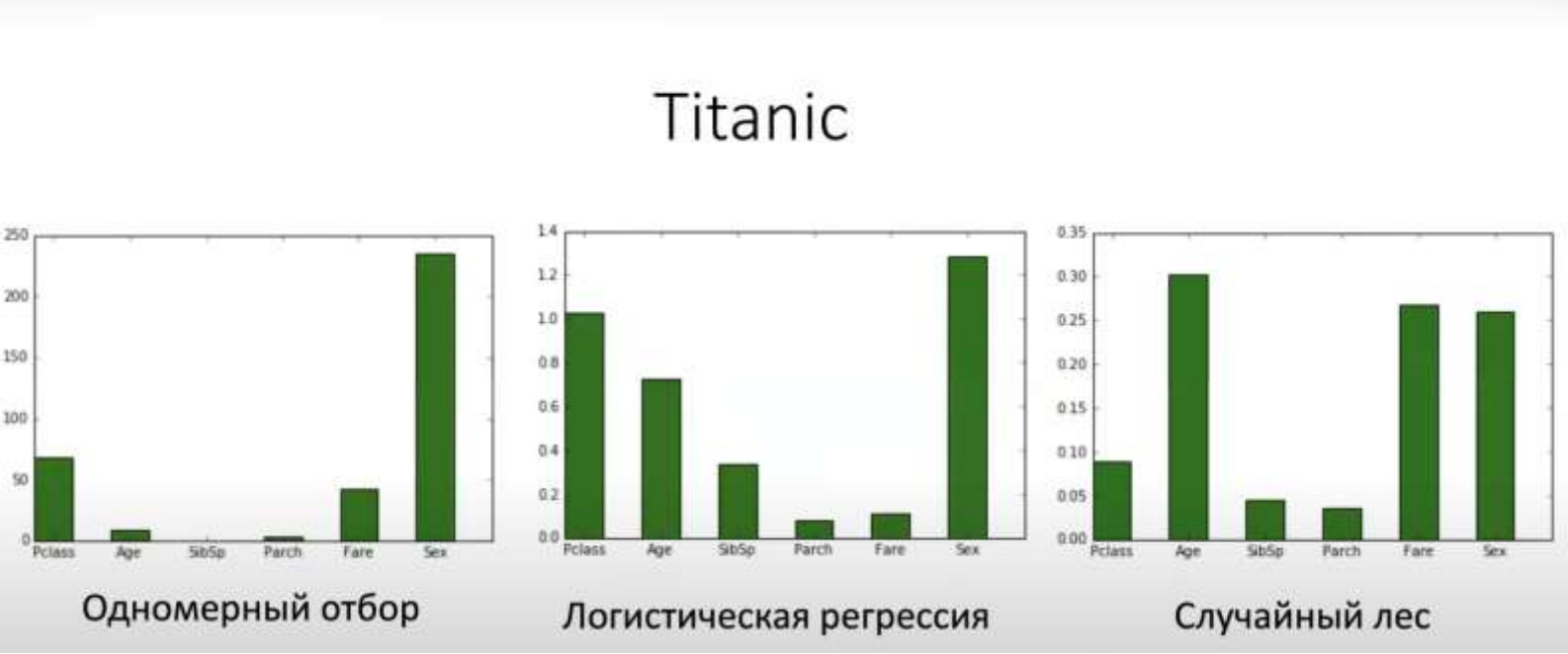
Недостатки: вычислительная сложность

Embedded methods

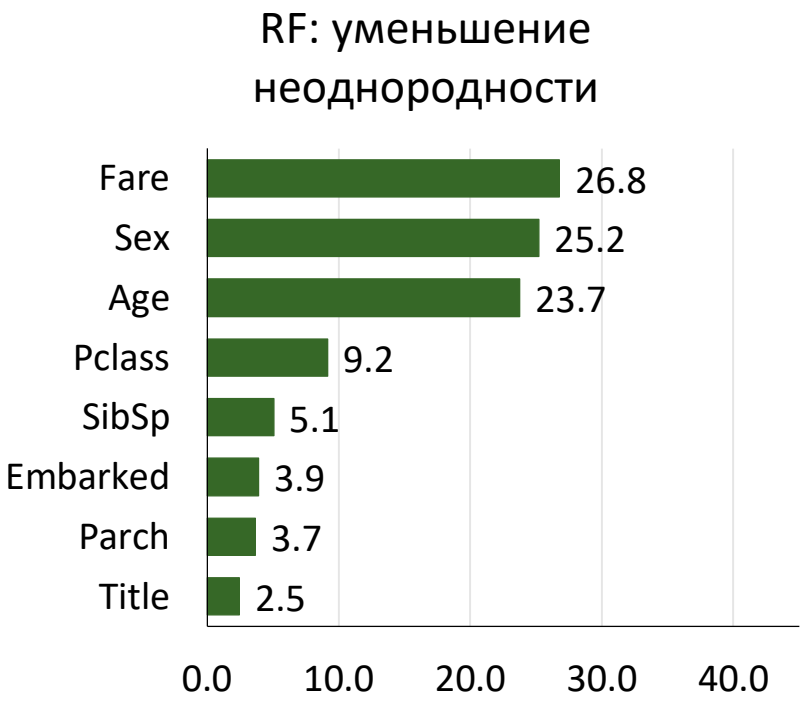
- *Встроенные методы отбора признаков* – отбор признаков на основе оценки важности признаков в процессе построения модели

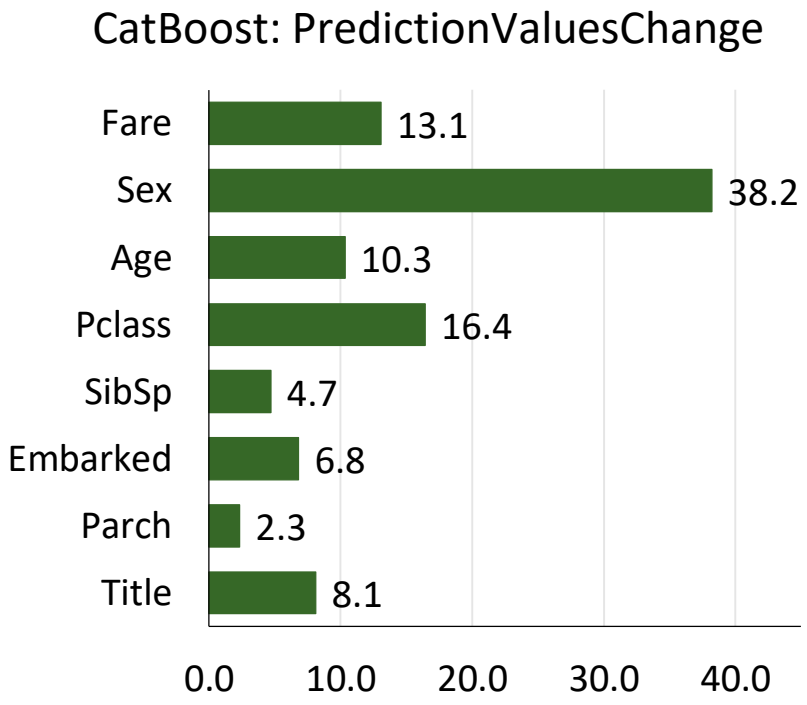
➢ Оценка значимости признаков выполняется вместе с обучением модели

➢ Зависят от модели – разные модели могут давать разную оценку значимости признаков



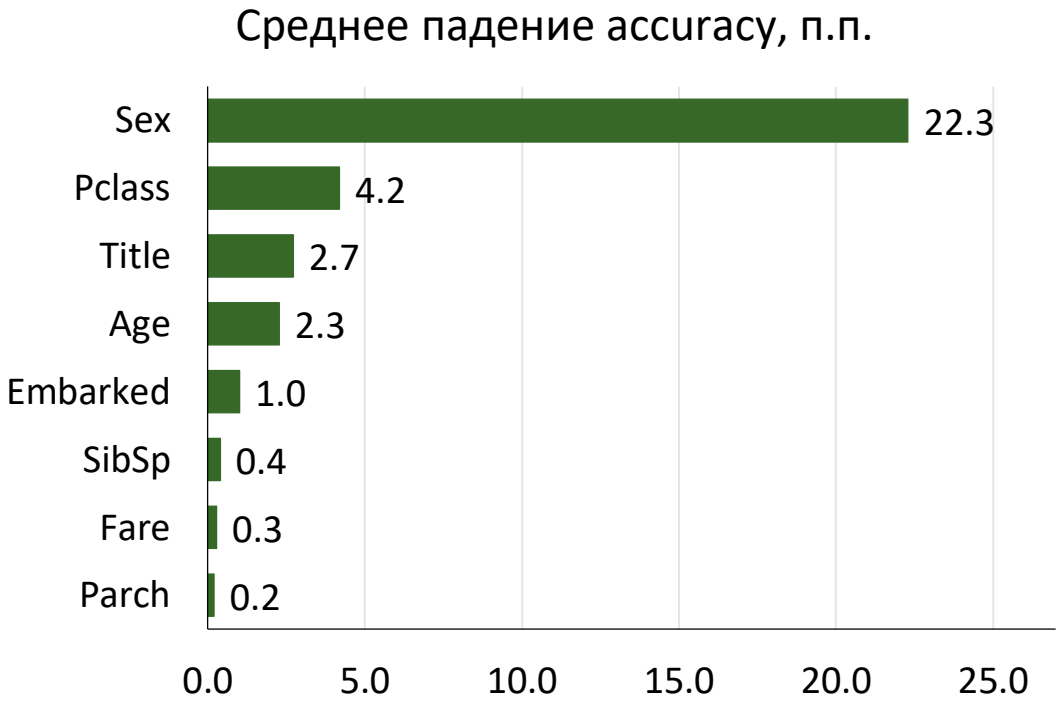
Встроенные важности: RF и CatBoost





RF суммирует вклад разбиений в уменьшение неоднородности.
CatBoost оценивает изменения ответов по значениям листьев.
Важности не доказывают причинное влияние.

Permutation importance



Перемешиваем один столбец validation.

Считаем тот же score без переобучения.

Повторяем N раз: оцениваем среднее и разброс падения.

Важность признака — среднее падение качества модели при случайном перемешивании его значений.

Чем сильнее падает качество, тем больше модель использует этот признак.

Коррелирующие признаки могут заменять друг друга, поэтому их индивидуальные важности могут быть низкими.

SHAP

SHAP (SHapley Additive exPlanations) объясняет предсказание для конкретного объекта через вклады его признаков.

Основа метода — значения Шепли из теории игр. Признаки выступают «участниками»: вклад каждого усредняет изменение ожидаемого ответа при добавлении его значения к разным наборам известных признаков.

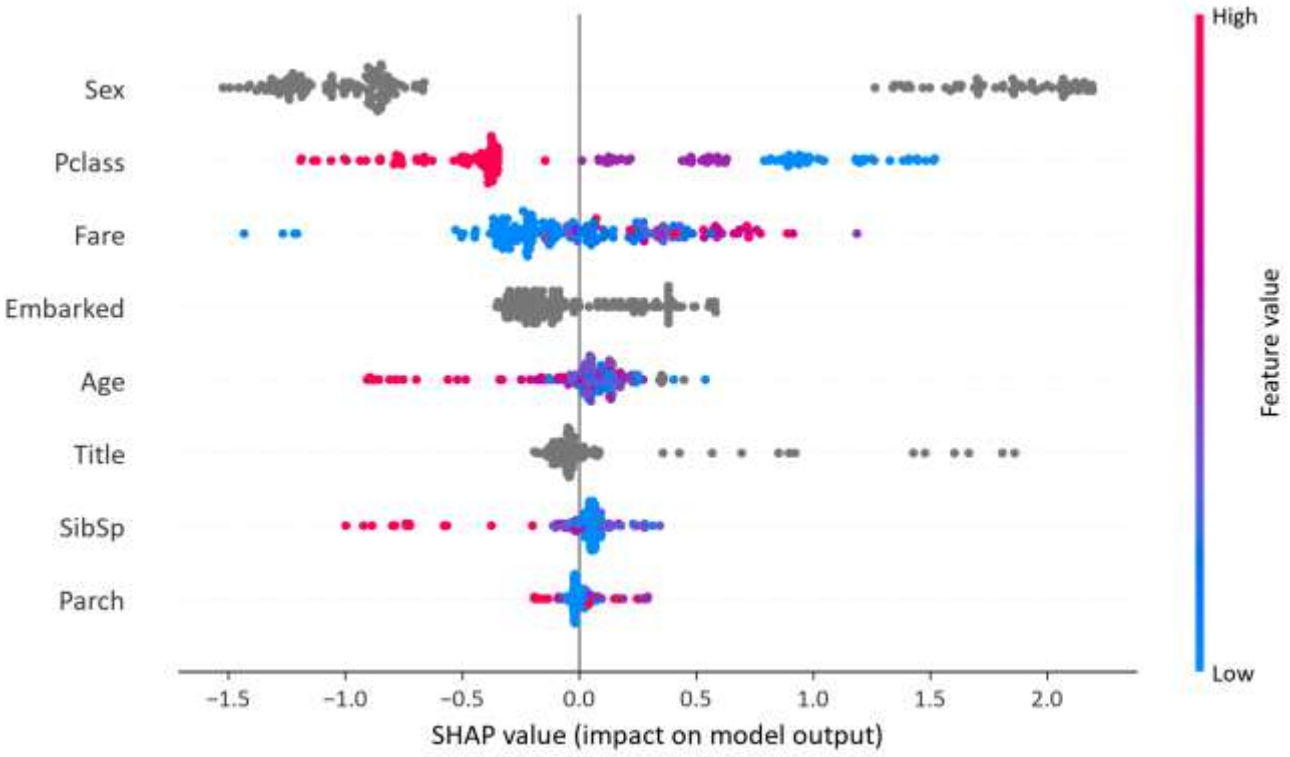
Ответ модели = базовый ответ + сумма SHAP-вкладов

Базовый ответ — ожидаемый выход модели в схеме объяснения.
Положительный вклад повышает ответ, отрицательный снижает.
Один признак может иметь разные вклады у разных объектов.

Метод SHAP используется для интерпретации модели – оценка важности признаков. SHAP описывает поведение модели, а не доказывает причинные связи.

SHAP

Beeswarm: каждая точка — пассажир validation.
Строки упорядочены по среднему модулю SHAP-вклада.



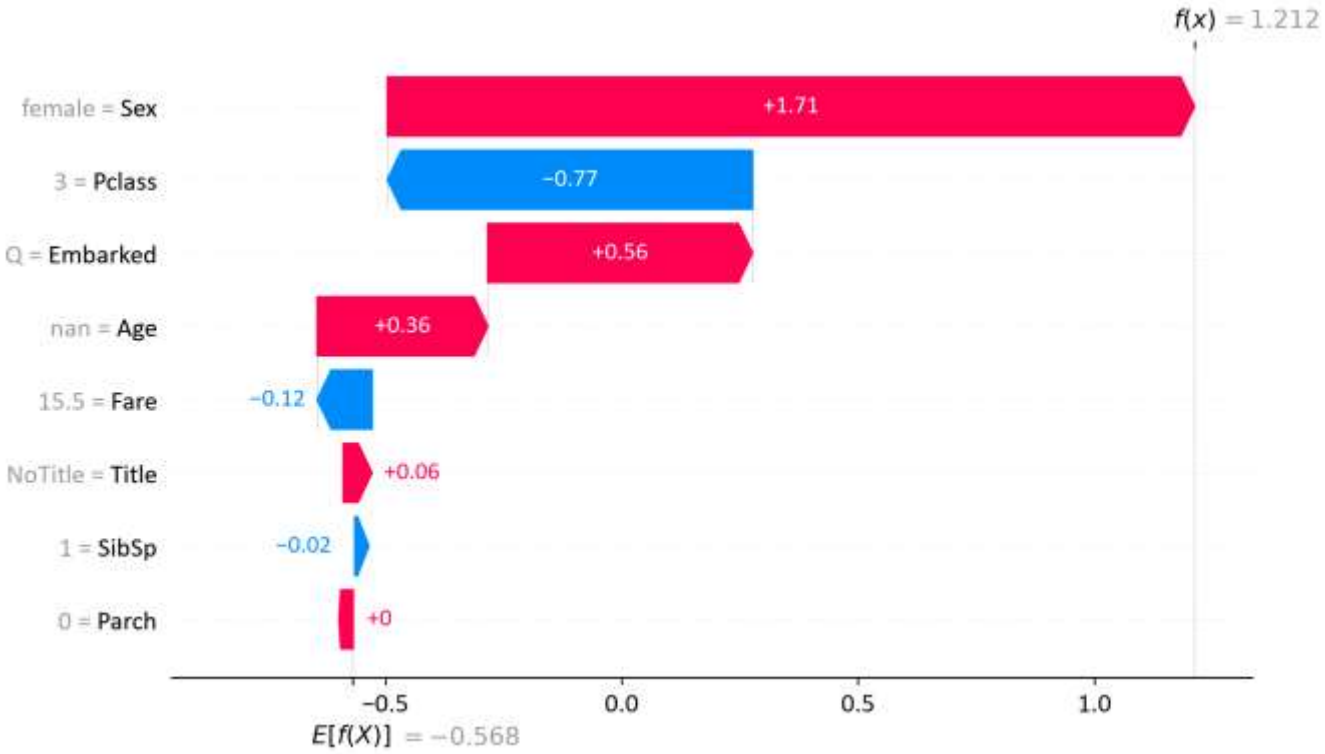
Справа от нуля: вклад повышает скор выживания.

Цвет чисел: красный — больше, синий — меньше.

Серый: категории или пропуски.

SHAP: объяснение одного предсказания

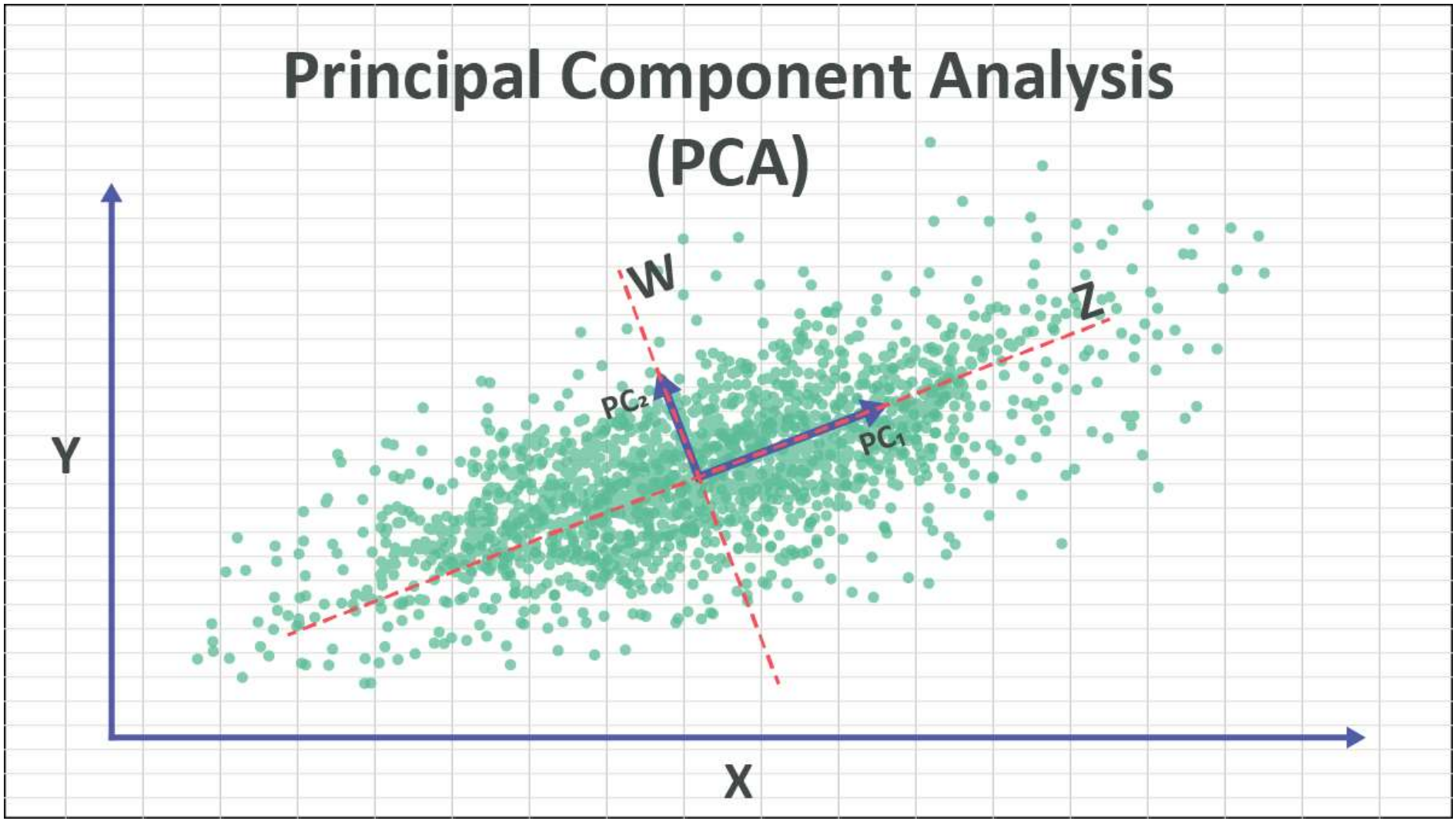
Пассажир 242: женщина, 3-й класс, Age отсутствует, Fare = 15,5.



Красный:
повышает скор выживания

Синий:
снижает скор

E[f(X)]:
базовый скор


Principal Component Analysis
(PCA)
W
Z
PC2
PC1
Y
X

Метод главных компонент

Основан на предположении о линейности отношений данных и их проекции на подпространство ортогональных векторов, в которых дисперсия будет максимальной. Такие вектора называются главными компонента и определяют направления наибольшей информативности данных.

Например, $z_1 = v_1^T x$: новый признак является взвешенной суммой исходных.

Реализация PCA через сингулярное разложение:

1. Центрирование данных и разложение центрированной матрицы:

$$X = U\Sigma V^T$$

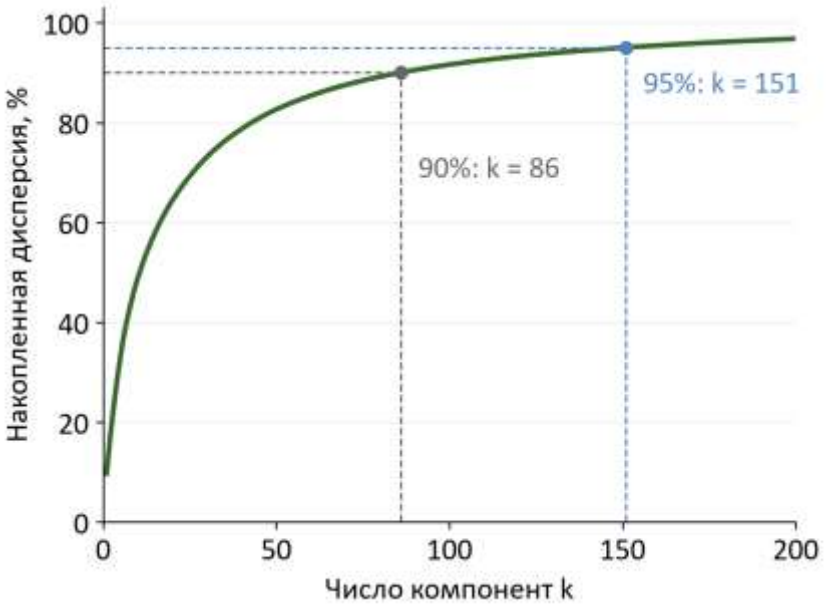
2. Выбор k направлений: выбираем $k \leq r$, так чтобы столбцы V соответствовали наибольшим сингулярным значениям, задавая направления компонент

3. Объясненная дисперсия для каждой главной компоненты вычисляется как возведенные в квадрат соответствующие сингулярные значения, разделенные на количество сэмплов:

$$\lambda_j = \sigma_j^2 / (\ell - 1).$$

MNIST: изображение 28 × 28 пикселей = 784 признака.
Сжимаем изображение до k компонент и восстанавливаем обратно.

Дисперсия: первые 200 компонент



Восстановление одной цифры

Исходное



5 компонент

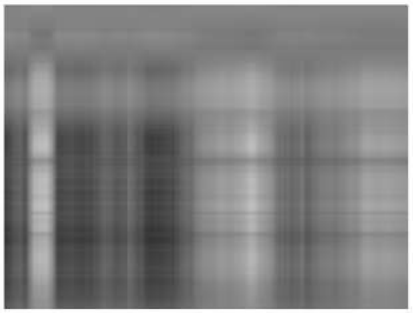


30 компонент



100 компонент





(a) 1 principal component



(b) 5 principal component



(c) 9 principal component



(d) 13 principal component



(e) 17 principal component



(f) 21 principal component



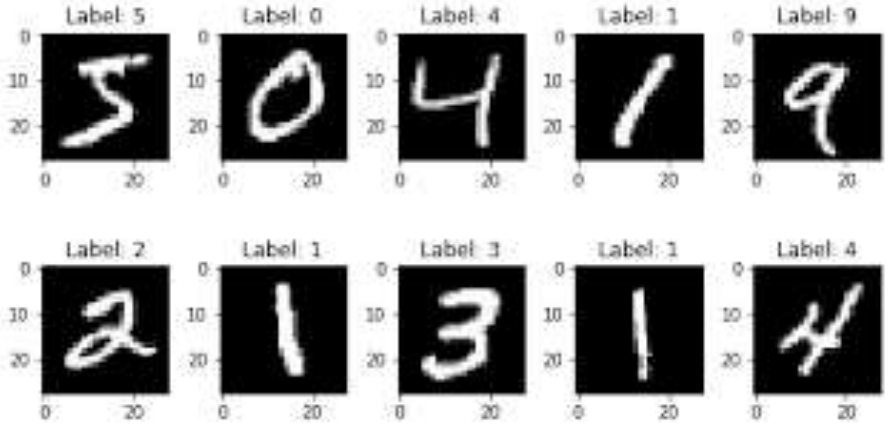
(g) 25 principal component

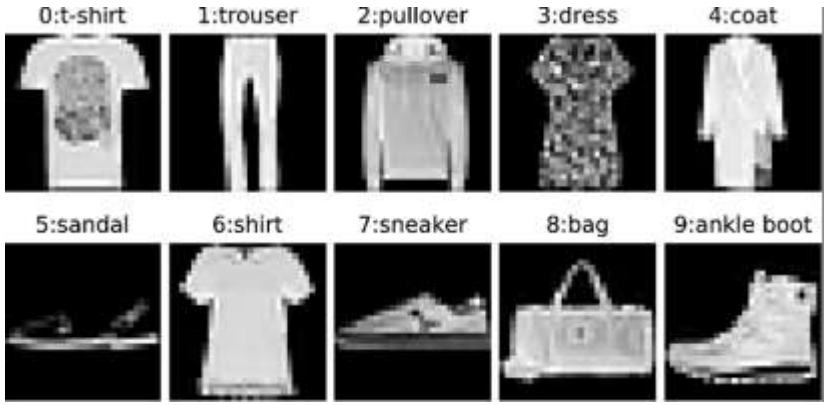


(h) 29 principal component

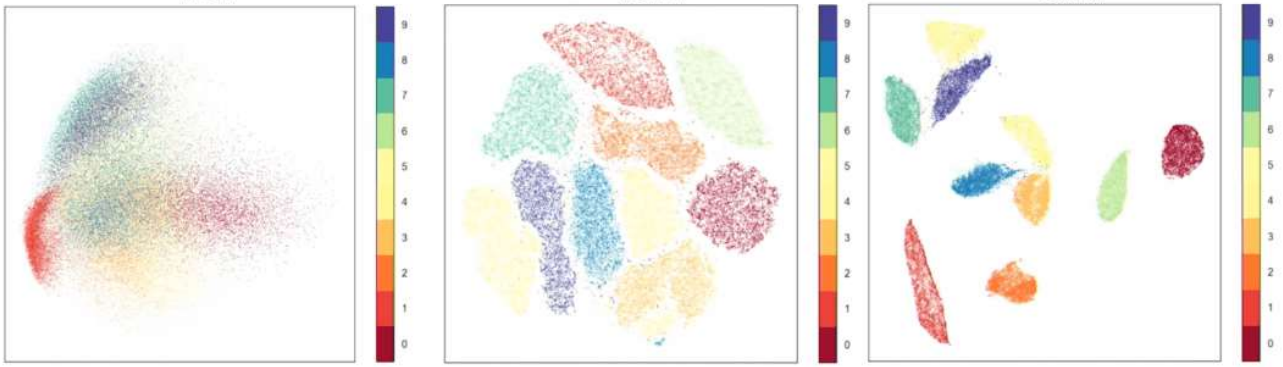
Другие методы понижения размерности

- *Locally Linear Embedding (LLE)* – алгоритм создания линейных комбинаций каждой точки из её соседей с последующим восстановлением этих комбинаций в пространстве более низкой размерности, что позволяет сохранить нелинейную геометрию данных и быть полезным для некоторых задач, где глобальные свойства менее важны. С другой стороны, такой подход имеет высокую вычислительную сложность и может быть чувствителен к шуму.
- *t-SNE (t-Distributed Stochastic Neighbor Embedding)* — алгоритм, который преобразует сходства между данными в вероятности и в дальнейшем пытается минимизировать расхождение между распределениями вероятностей в пространстве высокой и низкой размерности. t-SNE эффективен при визуализации данных высокой размерности, однако может искажать глобальную структуру данных, поскольку не учитывает линейные зависимости, а лишь их близость в исходном пространстве.
- *UMAP (Uniform Manifold Approximation and Projection)* — ещё один алгоритм, подходящий для визуализации данных, который основан на идеи, что данные лежат на некотором однородном многообразии, которое можно аппроксимировать с помощью графа соседей. Такой подход учитывает глобальную структуру данных и позволяет лучше адаптироваться к различным типам данных, а также лучше справляться с шумом и выбросами, чем t-SNE.
- *Автоэнкодеры* — тип нейронных сетей, основанный на обучении кодировщика преобразовывать входные данные в низкоразмерное представление, с последующим обучением декодера восстанавливать исходные данные из этого представления. Autoencoders могут также использоваться для сжатия данных, удаления шума и многих других целей.

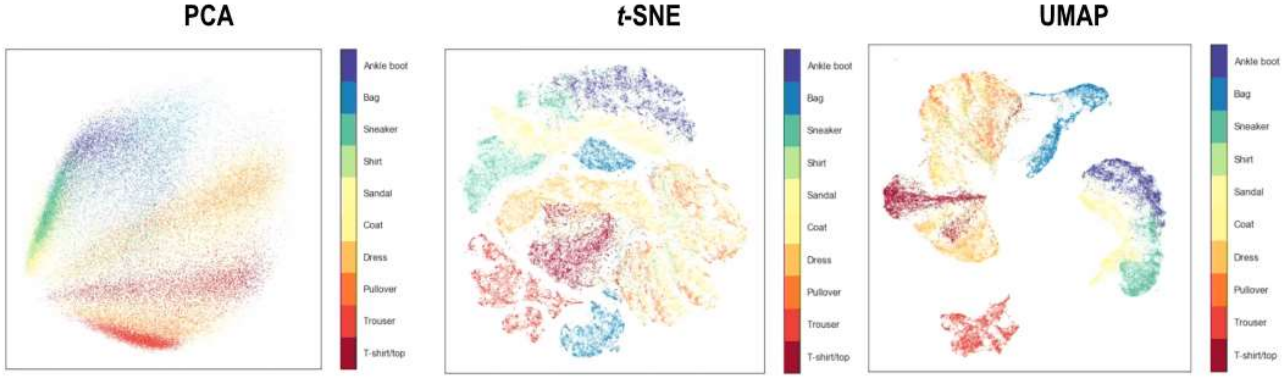




MNIST Digits



Fashion MNIST



Как выбрать подход

Цель	*С чего начать*
Сократить набор исходных признаков	Фильтры, RFE / RFECV. L1 / Elastic Net: линейные модели.
Найти релевантные признаки, включая избыточные	Boruta: поиск всех релевантных признаков по сравнению с шумом.
Понять, как модель использует признаки	Permutation: падение качества при перемешивании признака. SHAP: вклады в ответ модели.
Сжать данные в новые признаки	PCA: линейное сжатие. Для предсказания число компонент выбираем по CV.
Исследовать соседства и группы на карте	PCA как простой ориентир. t-SNE / UMAP: изучаем соседства.

Отбор признаков, оценка важности и визуализация решают разные задачи.

Как выбрать подход

1. Сравнить с исходной моделью:

Сравниваем с моделью без отбора и PCA: те же данные, фолды и метрика. Заполнение пропусков, масштабирование, отбор и PCA обучаем только на train. Число признаков и компонент подбираем по CV.

2. Проверить качество и устойчивость:

Смотрим среднюю метрику, разброс по фолдам и устойчивость отбора. Итог: отдельный test, не использованный при выборе метода и настроек. Альтернатива - внешние фолды вложенной CV. Коррелирующие признаки могут заменять друг друга.

3. Учесть стоимость (время):

Учитываем время, память, стоимость сбора и доступность признаков. Отбор может сократить число собираемых признаков.

Сопоставимое качество при меньших затратах — полезный результат.

Рост качества после отбора или понижения размерности не гарантирован.